

Support Ops Copilot

An LLM pipeline that classifies support complaints by category and escalation risk and generates a weekly report.

github.com/TobyPorterDay/support-ops-copilot

62%	92%	\$0.35	\$0.001
Precision (High-risk flags)	Recall (Genuine escalations caught)	Per report (Batches API, 600 complaints)	Per complaint (classification)

1 — The Problem

Customer support teams need a way to identify which tickets require manager escalation. That process involves manually reading each ticket and making a judgment call, taking up a considerable amount of time. The cost of a missed escalation is high — a churned account and the reputational damage that follows. This tool uses an LLM to make the preliminary escalation call.

2 — What I Tried First and Why It Didn't Work

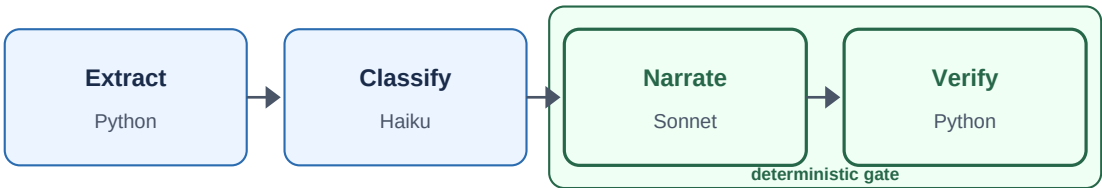
The first dataset I looked at was a customer support set from Kaggle. On inspection, the ticket narratives contained unfilled placeholders and synthetic filler sentences repeated throughout, so any eval would measure the text generator rather than the classifier. I needed real variation.

The Consumer Financial Protection Bureau (CFPB) maintains a public Consumer Complaint Database of real cases with all sensitive information redacted. Descriptions from real consumers provided the variation necessary to produce distinct classifications.

The initial taxonomy was built from 10 CFPB samples, but the dataset was organised by complaint type so the test sample was homogeneous. A resampled set with equal representation of each type in random order was used instead. Categories were revised across three versions: *Lack of action* was refined to a specific behaviour; two categories were added to cover additional patterns; *Predatory practices* was reclassified as a risk signal rather than a category, leaving five categories in total.

3 — What I Built

A four-stage pipeline. Stages 3 and 4 form a deterministic gate to prevent fabricated numbers from reaching the report.



Stage	Step	Description
1	Extract	Python script reads complaints from CSV.
2	Classify	Haiku assigns a category and escalation risk (Low / Medium / High) to each complaint.
3	Narrate ♦	Sonnet receives only the computed figures dict — no raw text — and writes the executive summary.
4	Verify ♦	Python audits every numeric token in the report against the figures dict; fails if any number cannot be sourced there.

♦ Deterministic gate

4 — The Eval

The taxonomy and risk rubric were committed to git (hash 34c6ff7) before hand-labelling began, preventing any post-hoc adjustment to match the model's output. The outcome column was removed from the spreadsheet during labelling to avoid anchoring bias — knowing whether the company provided relief would push toward labelling high.

The main challenge was over-escalation: the model initially rated almost all cases as High. The rubric was updated to cap speculative harm at Medium, reserving High for concrete, realised harm. Over-escalations dropped from 27 to 14; missed escalations rose from 0 to 2.

Metric	v2	v3 (Final)
Category exact match	54%	64%
Risk exact match	52%	59%
High-tier precision	52%	62%
High-tier recall	100%	92%
Over-escalations	27	14
Missed escalations	0	2

v2's perfect recall is misleading — it flagged almost everything High, so it caught every escalation by default. v3 accepts 2 missed escalations to halve the false-alarm rate, the correct trade for a tool a team would actually use.

5 — Cost and Latency

Total cost to classify a 600-complaint weekly run is ~\$0.71 (~\$0.001 per complaint). Prompt caching was considered but Haiku 4.5 requires a minimum of 4,096 tokens; the rubric is 508 tokens. Because the tool runs weekly, calls can be batched and run asynchronously using the Batches API, which gives a 50% discount — bringing the report cost to **\$0.35**.

6 — Limitations

- Running the full 600-complaint sample produced a 73% High-risk rate. Given 62% precision, the true rate is around 45% (0.73×0.62). The 73% figure is a review queue, not a precise total.
- The verifier detects numbers absent from the figures dict but does not catch value-swaps between two legitimate figures. Full claim-level verification requires sentence parsing and is a next step.
- The *Unable to make contact* category appeared in the full run but not in the 56-complaint eval sample, so no accuracy measurement exists for it.
- Labels reflect one person's judgment. Validation by an experienced customer operations manager is the real accuracy test.

7 — What I'd Do Next

- Add more Medium and High examples to the rubric with explanations — improves precision and pushes input token count past 4,096, enabling prompt caching and further reducing Haiku costs.
- Add sentence-level parsing to the verifier to catch misattributed values.
- Test Sonnet in the classifier stage to quantify any accuracy gain against the cost increase.
- Have a customer ops manager review the eval labels and assess whether the report output is actionable for a leader.

8 — What I Learned

- The best way to prevent fabrication is to limit the model's context, not rely on the prompt. Sonnet receives only the computed figures dict — no raw text.
- Testing the verifier by deliberately supplying incorrect values revealed its gaps immediately.
- The eval must be updated as the rubric changes; stale parameters produce misleading results.

9 — AI Use

Claude Code and a Claude chat were used throughout this project. I made the judgment calls, hand-labelled the 56 complaints used in the eval, and wrote this case study.
